Tools-A2A: Architecture & User Guide

Version: Sprint 3 (PostgreSQL, LXC, Peer Sync) **Date:** April 2026

Table of Contents

1. What Is Tools-A2A?
2. Architecture Overview
3. Protocol Stack
4. Component Reference
5. Database Schema
6. File Sync Engine
7. Group & Visibility Model
8. Agent Identity: Soul, Memory, Todos
9. Reconciler
10. Peer Gateway Sync
11. User Onboarding Flow
12. User Management
13. LXC Agent Lifecycle
14. Security Model
15. Configuration Reference
16. Running Locally
17. Multi-LAN Topology

What Is Tools-A2A?

Tools-A2A is a **Rust-native agent-to-agent communication stack** built on Google's A2A protocol (v0.3). It gives AI agents a standard way to:

- **Register** themselves and announce capabilities
- **Call** other agents by name, with policy enforcement (who can call whom, hop limits)
- **Carry identity** — every agent has a *soul* (personality), *memory* (append-only notes), and *todos* (tasks, never deleted)
- **Sync project files** across sites using content-addressed object storage
- **Federate** with other gateway instances via mTLS gRPC, with JWT auth at the peer boundary

The gateway is the nerve centre. Agents are thin — they register with the gateway, handle incoming calls, and delegate outbound calls through the gateway. The gateway enforces all policy.

Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│  Site A  (NAS / LAN — trusted zone)                                  │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐              │
│  │ orchestrator │  │   research   │  │    writer    │  Rust agents │
│  │  :8080       │  │  :8081       │  │  :8082       │              │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘              │
│         │  gRPC plain      │                 │                      │
```

```
│          │  (:7240, LAN)    │                 │                      │
│          └──────────────────┴─────────────────┘                      │
│                             │                                         │
│                ┌────────────▼──────────────────┐                     │
│                │        a2a-gateway            │                     │
│                │                               │                     │
│                │  :7240  plain gRPC  (agents)  │                     │
│                │  :7241  mTLS gRPC   (peers)   │                     │
│                │  :7242  HTTPS       (A2A spec)│                     │
│                │  :7243  HTTP        (web UI)  │                     │
│                │                               │                     │
│                │  PostgreSQL  ←── state        │                     │
│                │  ./data/objects ←── file blobs│                     │
│                └───────────────────────────────┘                     │
│                                │ mTLS + JWT                           │
└────────────────────────────────┼──────────────────────────────────────┘
                                 │ Cloudflare Tunnel
                        agents.cubic.build
                                 │
                     ┌───────────▼──────────────┐
                     │   Site B Gateway         │
                     │   (same architecture)    │
                     └──────────────────────────┘
```

Key design principles

Principle	Implementation
LAN is trusted	Port 7240: no TLS, no auth — the network boundary is the security boundary
Internet boundary is zero-trust	Port 7241: mTLS + short-lived JWT, peer name verified on handshake
Policy is gateway-side	Hop limits, call graph, and visibility checks all enforced by gateway, never by agents
Identity is persistent	Soul/memory/todos survive agent restarts; stored in PostgreSQL
Files are content-addressed	Object store mirrors git's loose-object layout — sha256, immutable, idempotent
Reconciler is rules-based	30-second loop assigns tasks by capability matching — no LLM required

---

Protocol Stack

```
┌──────────────────────────────────────────────────────────────┐
│  Layer 3 — Agent-to-Agent (cross-site)                       │
│  A2A v0.3  JSON-RPC 2.0  :7242  (spec-compliant)            │
│  gRPC fast-path            :7241  (our gateways, mTLS)       │
├──────────────────────────────────────────────────────────────┤
│  Layer 2 — Agent-to-Tool                                     │
```

```
│  MCP  (Model Context Protocol)  — agent-level              │
├──────────────────────────────────────────────────────────────┤
│  Layer 1 — Local Transport  (LAN)                           │
│  gRPC plain  :7240  agent ↔ gateway                         │
├──────────────────────────────────────────────────────────────┤
│  Layer 0 — Fan-out / Events                                 │
│  NATS Core  :4222  (optional, graceful degradation)         │
└──────────────────────────────────────────────────────────────┘
```

Port map

Port	Protocol	Direction	Purpose
7240	gRPC (plain)	LAN inbound	Agent registration, calls, identity RPCs
7241	gRPC (mTLS)	Internet inbound	Peer gateway handshake, delegation, file sync
7242	HTTPS	Internet inbound	A2A spec JSON-RPC, Agent Card, `/health`
7243	HTTP	LAN inbound	Web dashboard — projects, agents, contributions
4222	NATS	LAN outbound	Fan-out notifications to agents

Component Reference

a2a-gateway

The core binary. Owns all state. All other components are thin clients.

Module	Responsibility
`config.rs`	Load `gateway.toml`, env-var overrides, call graph validation
`db.rs`	PostgreSQL connection pool, run migrations on startup
`registry.rs`	In-memory agent cache (DashMap) backed by PostgreSQL
`router.rs`	Call graph enforcement + hop limit checks
`local_server.rs`	`:7240` gRPC — Register, Call, Stream, ListAgents, identity RPCs
`peer_server.rs`	`:7241` mTLS gRPC — Handshake, Delegate, file sync
`peer_client.rs`	Outbound calls to peer gateways
`http_server.rs`	`:7242` — A2A Agent Card, JSON-RPC handler, `/health`
`web/mod.rs`	`:7243` — REST API for web dashboard
`identity.rs`	Soul / memory / todo CRUD
`reconciler.rs`	30-second task reconciliation loop
`file_sync.rs`	Snapshot creation, diff, apply
`object_store.rs`	Content-addressed blob store

`projects.rs`	Project CRUD + visibility state machine
`groups.rs`	Group + invite management
`contributions.rs`	Change proposals + voting
`task_log.rs`	Immutable audit log of every delegated task
`lxc.rs`	Proxmox REST API client — list/spawn/stop containers
`nats_bus.rs`	NATS fan-out (optional)
`auth.rs`	JWT issue/validate for peer boundary

a2a-sdk

Library crate for writing Rust agents.

```
let agent = Agent::builder()
    .name("my-agent")
    .gateway("http://gateway:7240")
    .build()
    .await?;

agent.serve(|msg| async move {
    Ok(Response::text("hello"))
}).await?;

// Call another agent
let result = agent.call("research", json!({"query": "rust async"})).await?;
```

a2a-sidecar

Transparent Unix-socket → TCP gRPC bridge. Lets Python/JS agents connect to the gateway via a shared Docker volume without speaking gRPC natively.

```
Python agent
  └── /run/a2a/agent.sock  (Unix socket)
       └── a2a-sidecar  (Rust)
            └── gateway:7240  (TCP gRPC)
```

a2a-cli

Operator CLI. All management operations.

```
a2a-cli agents list
a2a-cli call research '{"query":"test"}'
a2a-cli identity show research --soul
a2a-cli identity show research --todos
a2a-cli peer add --name site-b --endpoint https://agents.site-b.example
a2a-cli lxc list
```

```
a2a-cli lxc spawn research
a2a-cli health
```

Database Schema

All state lives in PostgreSQL. The gateway runs migrations on startup — no manual schema setup needed.

Migration 001 — Core identity

```
agents           name (PK), version, endpoint, capabilities (JSON), soul_toml,
last_seen, registered_at
memories         agent_name (PK → agents), content_md (append-preferred)
todos            id (BIGSERIAL), agent_name, task, status, notes, created_at,
completed_at
requests         id (BIGSERIAL), content, created_at
project_todos    id (BIGSERIAL), task (UNIQUE), assigned_to, status, notes,
source_todos (JSON), timestamps
```

Migration 002 — Projects

```
projects         id (UUID), name, repo_url, description, visibility, folder_path,
file_glob,
                 is_active, onboarding_step, origin_gateway, peer_gateway,
peer_project_id
agent_projects   (agent_name, project_id) composite PK, joined_at
```

Migration 003 — File sync

```
objects          sha256 (PK), size_bytes, created_at
snapshots        id (sha256), project_id, gateway, transcript, created_at
project_heads    project_id (PK), snapshot_id → snapshots, updated_at
```

Migration 004 — Task log

```
task_log         id (BIGSERIAL), trace_id, caller, target, method, payload_sha256,
                 todo_id → todos, status, created_at, completed_at, error
```

Migration 005 — Contributions

```
contributions    id (UUID), project_id → projects, author_gateway, title, description,
                 snapshot_id, diff_summary, status, vote_threshold, created_at
votes            (contribution_id, gateway_name) PK, vote (approve/reject), voted_at,
comment
```

Migration 006 — Groups

```
groups           id (UUID), name (UNIQUE), description, created_by (gateway name),
created_at
```

```
group_members    (group_id, gateway_name) PK, joined_at, invited_by
invites          token (32-byte hex PK), group_id, created_by, created_at, expires_at
(48h TTL),
                 used_at, used_by
group_projects   (group_id, project_id) PK, added_at
```

File Sync Engine

Projects sync files using a content-addressed object store that mirrors git's loose-object layout.

```
Snapshot creation (local project):

  folder_path/
    ├── a.rs   ── sha256 → objects/a3/f9...
    ├── b.rs   ── sha256 → objects/7c/22...
    └── lib.rs ── sha256 → objects/d1/b8...
                    │
                    ▼
             transcript (sorted)
             F|a.rs|a3f9...|1024|644|1712345678
             F|b.rs|7c22...|2048|644|1712345679
             F|lib.rs|d1b8...|512|644|1712345680
                    │
                    ▼  sha256(transcript)
             snapshot.id = "e4a2..."
             stored in snapshots table
             project_heads updated
```

```
Peer receives snapshot notification:

  1. Peer calls diff_against_head() → SnapshotDiff { added, removed, modified }
  2. For each added/modified: fetch object bytes if not already present
  3. apply_peer_snapshot() writes snapshot row, advances project_heads
  4. NATS publishes notification to local agents
```

Diff structure

Field	Meaning
`added`	Files in new snapshot not in old
`removed`	Files in old snapshot absent from new
`modified`	Same path, different sha256
`required_objects()`	Hashes the peer must fetch

Group & Visibility Model

Projects have three visibility levels, controlled by a state machine.

```
lan-only ──────► group ──────► public
   ◄──────────────────────────────────
      (one level at a time, project must be idle)
```

Visibility	Who can see / sync	Typical use
`lan-only`	Only this gateway's agents	Early dev, private work
`group`	All gateways in the group	Trusted team collaboration
`public`	Any gateway (open internet via :7242)	Open source projects

Group membership flow

```
Alice (site-a) creates group "alpha-team":
  POST /groups  →  group_id returned
  Alice auto-joins as first member

Alice invites Bob (site-b):
  POST /groups/{id}/invites  →  token (64-char hex, 48h TTL)
  Alice sends token to Bob out-of-band (Signal, email, etc.)

Bob joins:
  POST /groups/{id}/join  body: {"token": "..."}
  Token consumed atomically — cannot be reused
  Bob's gateway added to group_members
  Bob now receives sync events for all group-visibility projects in this group
```

Invite tokens are 32 random bytes (hex-encoded), single-use, expire after 48 hours. There is no "pending invite" state — either used or expired.

Agent Identity: Soul, Memory, Todos

Every agent that registers gets a persistent identity row in PostgreSQL.

Soul (immutable personality)

TOML document defining the agent's role, capabilities, and constraints. Set at registration. The gateway preserves the original soul even if the agent re-registers (it only overwrites if the existing soul is empty).

```toml
[soul]
name = "research"
role = "Deep research assistant"
capabilities = ["web-search", "summarise", "cite-sources"]

[constraints]
max_hops = 2
allowed_targets = ["writer", "fact-checker"]
```

Memory (append-preferred markdown)

Free-form markdown document. Agents append context across sessions — current work, learned facts, recent decisions. The `UpdateMemory` RPC appends a timestamped section, never overwrites.

```
## 2026-04-07T14:22:00Z
Completed research on Rust async runtimes. Key finding: tokio's work-stealing
scheduler is optimal for mixed I/O + CPU workloads above ~50 concurrent tasks.
```

Todos (annotated, never deleted)

Tasks are created via `AppendTodo`. Completion annotates the row (`completed_at` , `notes`) — rows are never deleted. This gives a full audit trail of what each agent worked on.

```
Status machine:
  pending ──► in_progress ──► done
                  │
                  └──► (notes field records what was learned/produced)
```

Reconciler

The reconciler runs every 30 seconds in the gateway. It is purely rules-based — no LLM involved.

```
Reconciler loop:

  1. Load all agent todos WHERE status != 'done'
  2. Load all project_todos WHERE status = 'pending' AND assigned_to IS NULL
  3. For each unassigned project task:
       a. Parse required capabilities from task text
       b. Find agent whose soul.capabilities intersects requirements
       c. Assign: UPDATE project_todos SET assigned_to = agent_name
  4. Detect duplicates: same task text in 2+ agent todo lists → merge into one
project_todo
  5. Detect conflicts: two agents assigned mutually exclusive tasks → flag status =
'conflict'
  6. Write todo.md to data_dir (human-readable reconciled view)
  7. Emit tracing span with reconcile_run metrics
```

The reconciler does not communicate with agents directly. Agents poll their own todos via `ListTodos` RPC at startup.

Peer Gateway Sync

Two gateways establish a peer relationship over mTLS gRPC (:7241).

```
Handshake:
  Site A ──Handshake(name="site-a", pubkey)──► Site B
  Site B ──Handshake(name="site-b", pubkey)──► Site A
  Both sides verify the other's JWT
  Each side stores the peer in its registry

Periodic sync (PeerSyncCoordinator, every 60s per peer):
```

```
  1. For each group-or-public project:
       a. Call peer's GetHead(project_id)
       b. If peer head != local head:
            - Compute diff
            - Fetch missing objects from peer via GetObject RPCs
            - apply_peer_snapshot()
  2. Forward any pending task_log entries for tasks delegated to this peer
  3. Sync group membership updates

Delegation flow:
  Site A agent calls target on Site B:
    local_server receives Call(target="site-b::writer")
    → router detects cross-gateway target ("::" separator)
    → peer_client.Delegate(DelegateRequest) to site-b:7241
    → site-b routes to writer agent
    → response forwarded back through the chain
    → task_log records the round-trip
```

User Onboarding Flow

New local project

```
1. User runs:
   a2a-cli projects create \
     --name my-app \
     --repo https://github.com/user/my-app

2. Gateway creates project row:
   visibility = 'lan-only'
   onboarding_step = NULL  (local project, no onboarding needed)
   folder_path = ''        (set in step 3)

3. User sets the folder to watch:
   a2a-cli projects set-folder <id> /home/user/projects/my-app

4. File sync runs automatically:
   - scan_and_snapshot() walks the folder
   - All files stored as content-addressed objects
   - Snapshot committed, project_heads updated
   - NATS notification sent to any interested agents

5. Assign agents to the project:
   a2a-cli projects add-agent <id> orchestrator
   a2a-cli projects add-agent <id> research
```

Peer project (receiving from another gateway)

```
1. Site A creates a project and wants Site B to mirror it.
   Site A sends a "project invite" out-of-band (the project UUID + gateway name).
```

```
2. Site B operator runs:
   a2a-cli projects accept-peer \
     --from site-a \
     --peer-project-id <uuid>

3. Gateway creates a project row:
   onboarding_step = 'awaiting_folder'
   peer_gateway = 'site-a'
   peer_project_id = <uuid>

4. Onboarding step 1 — folder placement:
   a2a-cli projects set-folder <id> /data/sync/my-app
   → gateway calls set_onboarding_step('awaiting_files')

5. Onboarding step 2 — file filter (optional):
   a2a-cli projects set-glob <id> "**/*.rs"
   → gateway calls set_onboarding_step(NULL)   # done

6. Peer sync coordinator picks up the project on next 60s tick:
   - Fetches snapshot from site-a
   - Downloads missing objects
   - Applies snapshot to local project_heads
   - Files appear on disk at folder_path

Onboarding state machine:
  NULL ◄──────────────────────────────────────────────────────────
                                                                  │
  awaiting_folder ──► awaiting_files ──► building_context ──►┘
        │                    │                  │
     (set folder)        (set glob)       (agent analyses)
```

User Management

Tools-A2A does not have "users" in the traditional sense — the unit of identity is the **gateway**, not a human user account. Access is managed at the gateway boundary.

Who can do what

Actor	Access model
LAN operator	Runs `a2a-cli` locally — full access via :7240
Peer gateway	mTLS cert + JWT — access limited to delegated calls and file sync
External A2A agent	Bearer token or API key declared in Agent Card — access via :7242
Web dashboard user	HTTP :7243 — currently unauthenticated (LAN only, TODO)

Gateway identity

Each gateway has:

- A **name** (`gateway.name` in `gateway.toml`) — unique across the federation

- A **JWT secret** (`A2A_JWT_SECRET` env var) — used to sign short-lived (1hr) peer tokens
- Optionally, a **TLS cert** (`cert_path` in peer config) — for mTLS on the :7241 peer channel

Group-based access control

Group membership determines which projects a peer gateway can sync:

```
project.visibility = 'group'
  → gateway must be a member of the group that owns this project
  → membership verified on every sync request

project.visibility = 'public'
  → any gateway may read (no membership check)
  → write (contribution) requires voting quorum
```

Invite lifecycle

```
create_invite(group_id, created_by)
  → generates 32-byte random token
  → stored with expires_at = now + 48h
  → token printed to CLI / returned via API

consume_invite(token, joining_gateway)
  → validates: not used, not expired
  → marks used_at, used_by atomically
  → adds joining_gateway to group_members
  → returns group (confirms which group was joined)
```

Expired or used tokens return a clear error: `"invite token is expired or already used"` .

LXC Agent Lifecycle

The gateway can spawn, start, and stop Proxmox LXC containers running agent images.

```
a2a-cli lxc spawn research

  1. LxcClient.spawn("research"):
       a. GET /api2/json/nodes/{node}/lxc  → list existing VMs
       b. new VMID = max(existing VMIDs) + 1  (floor 200)
       c. POST /api2/json/nodes/{node}/lxc   → clone from agent_template (9001)
       d. POST /api2/json/nodes/{node}/lxc/{vmid}/status/start
       e. Return LxcVm { vmid, name, status, mem, cpus }

  2. Container boots, starts agent binary:
       A2A_GATEWAY=http://gateway:7240
       A2A_ENDPOINT=http://{vmid-ip}:8080

  3. Agent registers with gateway via Register RPC
  4. gateway.upsert(AgentEntry{...})  → appears in agent list
```

```
Lifecycle:
  template (9001) → spawn → running → stop → (template unchanged)
                      │
                      └── agent registers → appears in registry
                                          → reconciler can assign tasks
```

Security Model

```
┌──────────────────────────────────────────────────────────────────┐
│  :7240 — LAN (trusted)                                           │
│  No TLS. No auth. The network segment is the security boundary.  │
│  Gateway enforces: call_graph edges, max 3 hops.                 │
│  hop_count is set and incremented by gateway only — callers      │
│  cannot spoof it.                                                │
├──────────────────────────────────────────────────────────────────┤
│  :7241 — Peer (zero-trust)                                       │
│  rustls mTLS: both sides present cert. Handshake verifies names. │
│  JWT Bearer in gRPC metadata: HS256, 1-hour TTL.                 │
│  Peer names are verified — a gateway cannot impersonate another. │
├──────────────────────────────────────────────────────────────────┤
│  :7242 — External / Cloudflare                                   │
│  CF-Access-Client-Id/Secret at the tunnel edge.                  │
│  Third-party A2A agents: Bearer token declared in Agent Card.    │
│  A2A spec JSON-RPC: SendMessage, GetTask, CancelTask.            │
├──────────────────────────────────────────────────────────────────┤
│  :7243 — Web UI                                                  │
│  Currently LAN-only HTTP. Auth TODO (tracked in TODOS.md).       │
└──────────────────────────────────────────────────────────────────┘
```

Call graph enforcement

```toml
# gateway.toml — who can call whom
[call_graph]
orchestrator = ["research", "writer", "reviewer"]
research     = ["writer", "fact-checker"]
writer       = ["reviewer"]
reviewer     = ["research"]

[call_graph.limits]
max_hops = 3
```

A call from `writer → orchestrator` is rejected at the gateway with `NotPermitted`. An anonymous external caller (empty caller string) bypasses call-graph checks — it can reach any registered agent, but still hits the hop limit.

Credential hygiene

All secrets come from environment variables — never from `gateway.toml`:

Secret	Env var
JWT signing key	`A2A_JWT_SECRET`
PostgreSQL password	`POSTGRES_PASSWORD`
PostgreSQL URL	`DATABASE_URL` (overrides `[db].url` in config)
Proxmox API token	`A2A_PROXMOX_TOKEN`
Cloudflare service token	`A2A_CF_CLIENT_ID` / `A2A_CF_CLIENT_SECRET`

Configuration Reference

gateway.toml

```toml
[gateway]
name        = "site-a"          # unique name for this gateway
port_local  = 7240              # plain gRPC — LAN agents
port_peer   = 7241              # mTLS gRPC — peer gateways
port_http   = 7242              # HTTPS — A2A spec + health
port_web    = 7243              # Web dashboard
data_dir    = "./data"          # object store + reconciler output files
reconcile_interval_secs = 30   # reconciler cadence

[db]
# DATABASE_URL env var takes precedence over this value.
url = "postgres://a2a:a2a@postgres/a2a"

[lxc]                           # optional; omit to disable LXC
proxmox_url    = "https://192.168.4.100:8006"
node           = "proxmox"
agent_template = 9001
network_bridge = "vmbr0"
# A2A_PROXMOX_TOKEN env var — never in this file

[[agents]]                      # seed agents (auto-upserted at startup)
name     = "orchestrator"
endpoint = "http://orchestrator:8080"

[call_graph]                    # DAG of allowed calls
orchestrator = ["research", "writer"]
research     = ["writer"]

[call_graph.limits]
max_hops = 3

[[peers]]                       # peer gateways (optional)
name      = "site-b"
```

```
endpoint  = "https://agents.site-b.example"
cert_path = "./certs/site-b.pem"
```

Running Locally

Prerequisites

- Docker + Docker Compose
- `cargo` (Rust stable)
- `grpc_health_probe` (for healthcheck in compose)

Quick start

```
# 1. Clone and configure
git clone https://github.com/your-org/tools-a2a
cd tools-a2a
cp .env.example .env
# Edit .env — set POSTGRES_PASSWORD and A2A_JWT_SECRET

# 2. Start the stack
docker compose up -d

# 3. Check health
cargo run -p a2a-cli -- health
curl http://localhost:7242/health

# 4. List registered agents
cargo run -p a2a-cli -- agents list

# 5. Call an agent
cargo run -p a2a-cli -- call research '{"query":"hello world"}'

# 6. View traces
open http://localhost:16686   # Jaeger UI

# 7. View NATS monitoring
open http://localhost:8222
```

Running tests

Tests require a real PostgreSQL instance:

```
# Start postgres only
docker compose up -d postgres

# Export connection URL
export DATABASE_URL=postgres://a2a:yourpassword@localhost/a2a

# Run all tests
cargo test --workspace
```

Multi-LAN Topology

Multiple gateways on the same LAN are supported — each needs its own `gateway.toml` with unique ports and its own PostgreSQL database.

```
LAN (192.168.1.x)
  ├── gateway-a  :7240/:7241/:7242/:7243   db: a2a_site_a
  │     ├── orchestrator :8080
  │     └── research     :8081
  │
  └── gateway-b  :7340/:7341/:7342/:7343   db: a2a_site_b
        ├── writer    :8180
        └── reviewer  :8181
```

To allow gateway-A agents to call gateway-B agents, add site-b as a peer in `gateway-a/gateway.toml`:

```toml
[[peers]]
name     = "gateway-b"
endpoint = "http://192.168.1.x:7341"
# cert_path optional for same-LAN peers (TODO: plain-gRPC peer mode)
```

Cross-gateway calls use the `::` separator convention:

```
a2a-cli call gateway-b::writer '{"draft":"..."}'
```

Note: Each gateway operates its own independent PostgreSQL database. The peer sync protocol handles cross-gateway state sharing when needed — there is no shared database between gateways.

Appendix: Key Data Flows

Agent registration

```
Agent binary starts
  │
  ├── reads A2A_GATEWAY env  →  "http://gateway:7240"
  ├── reads A2A_ENDPOINT env →  "http://this-agent:8080"
  │
  └── Register(AgentManifest) ──gRPC──► gateway :7240
        name, version, endpoint, capabilities[], soul_toml
              │
              ▼ gateway
      registry.upsert(AgentEntry)
        → INSERT INTO agents ON CONFLICT UPDATE
        → INSERT INTO memories (empty) ON CONFLICT DO NOTHING
      returns RegisterAck { gateway_name }
```

Task delegation chain

```
User request → requests table
    │
    ▼ reconciler (30s)
project_todos: assign "research X" → research agent
    │
    ▼ research agent polls ListTodos
todo: "research X"  status=pending
    │
    ▼ research calls gateway
Call(target="writer", method="draft", payload={...})
    │
    ▼ router checks
  hop_count + 1 ≤ 3 ?       → yes
  call_graph["research"]["writer"] ? → yes
  registry.get("writer") ?   → endpoint: "http://writer:8080"
    │
    ▼ gateway forwards HTTP to writer agent
    │
    ▼ response returns
task_log entry written (trace_id, caller, target, status, latency)
OTel span closed
```

Generated from sprint 3 source. For the latest schema run `psql $DATABASE_URL` *and* `\dt` *.*